

Single Center Problem in Location Optimization: Model and Implementation with Python

April 28, 2025

1 Package

Declare the package path contained:

```
[1]: import sys
     sys.path.append('ipynb/location/packages')
```

1.1 Graph Visualization

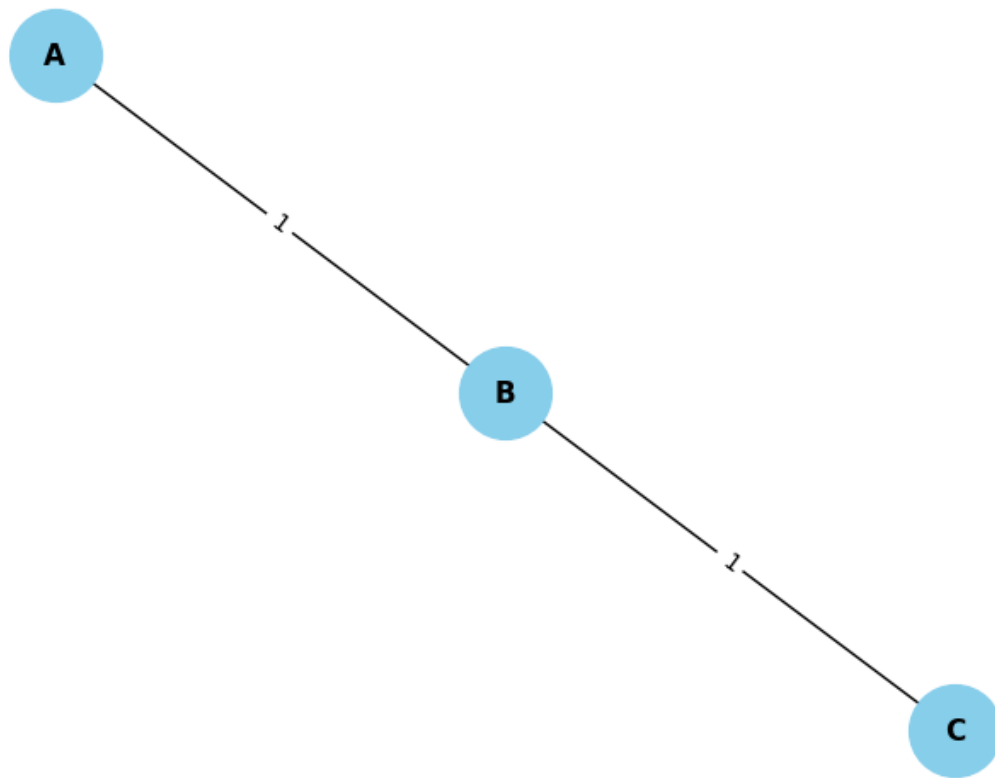
Source referred: [NetworkX](#) Declaring

```
[2]: from packages.GraphView import Graph
```

Example

```
[3]: matrix = [
      [0, 1, 0],
      [1, 0, 1],
      [0, 1, 0]
    ]

    G = Graph(matrix)
    G.show()
```



1.2 Shortest-path

Source referred: [GeeksforGeeks](#) Declaring

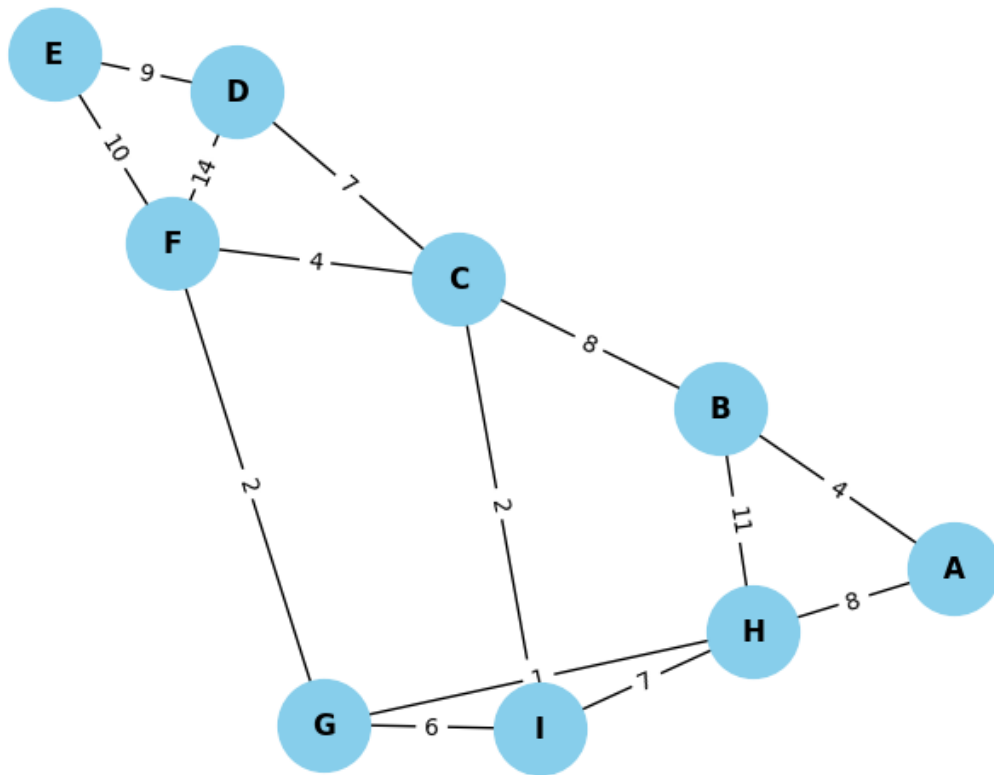
```
[4]: from packages import ShortestPath
```

Example Graph parameter

```
[5]: g = ShortestPath.Graph(9)
g.graph = [[0, 4, 0, 0, 0, 0, 0, 8, 0],
            [4, 0, 8, 0, 0, 0, 0, 11, 0],
            [0, 8, 0, 7, 0, 4, 0, 0, 2],
            [0, 0, 7, 0, 9, 14, 0, 0, 0],
            [0, 0, 0, 9, 0, 10, 0, 0, 0],
            [0, 0, 4, 14, 10, 0, 2, 0, 0],
            [0, 0, 0, 0, 0, 2, 0, 1, 6],
            [8, 11, 0, 0, 0, 0, 1, 0, 7],
            [0, 0, 2, 0, 0, 0, 6, 7, 0]]
```

Visualization

```
[6]: Visualization = Graph(g.graph)
Visualization.show()
```



Input 2 point requested

```
[7]: start = 'D'
end = 'A'
```

Execute

```
[8]: startInput = ord(start) - 65
endInput = ord(end) - 65
path, distance = g.getShortestPath(startInput, endInput)
result = [chr(65 + i) for i in path]
print(result)
print("Total distance:", distance)
```

```
['D', 'C', 'B', 'A']
Total distance: 19
```

2 Introduction

2.1 Bottleneck point

2.1.1 Description

For $v \in V$, $x = (e, \alpha) \in G$, the bottleneck point v on $e = (v_i, v_j)$ is

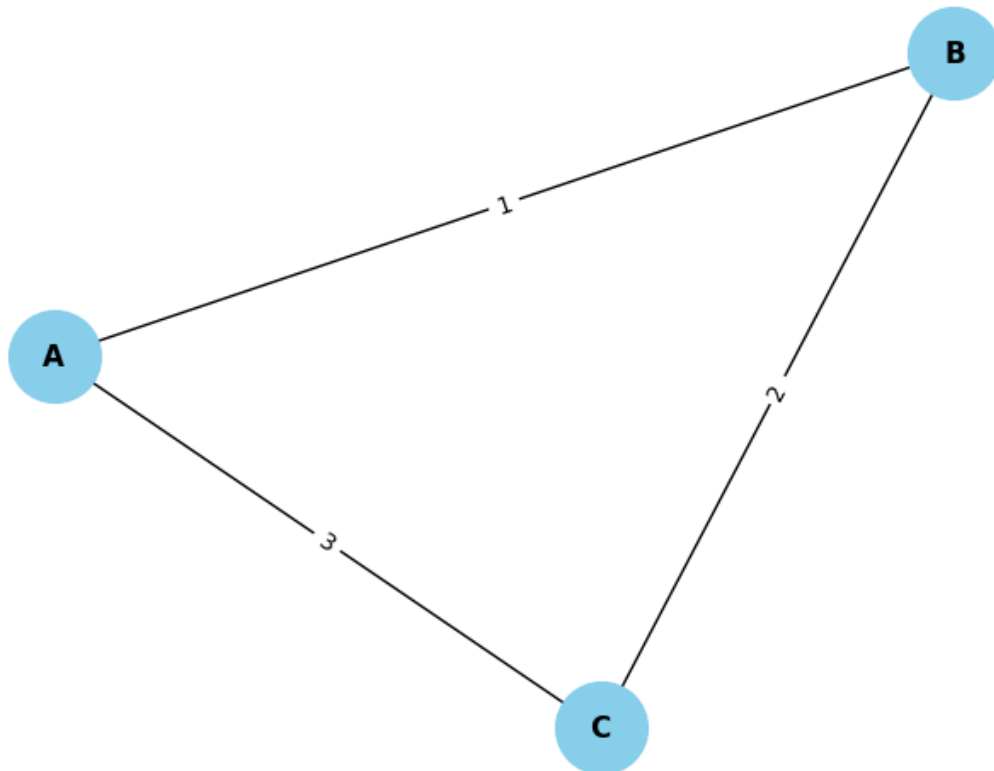
$$\alpha l_e + d(v_i, v) = (1 - \alpha) l_e + d(v_j, v)$$

- BP_e : Set of all bottleneck points on $e \in E$.
- BP : Set of all bottleneck points on G .

2.1.2 Example

```
[9]: graph = [  
    [0, 1, 3],  
    [1, 0, 2],  
    [3, 2, 0]  
]
```

```
[10]: ShowGraph = Graph(graph)  
ShowGraph.show()
```



```
[11]: from packages.Point import Local
```

```
[12]: a = Local(graph, 3, 0, 2)
      print(a.BottleNeck(1))
```

0.6666666666666666

2.2 Equilibrium point

2.2.1 Description

For $v_k, v_l \in V$, $v_k \neq v_l$, $x = (e, \alpha) \in G$, $e = (v_u, v_v)$, the equilibrium point is

$$\omega_k(\alpha l_e + d(v_k, v_u)) = \omega_l((1 - \alpha)l_e + d(v_l, v_v))$$

If $x \in G$

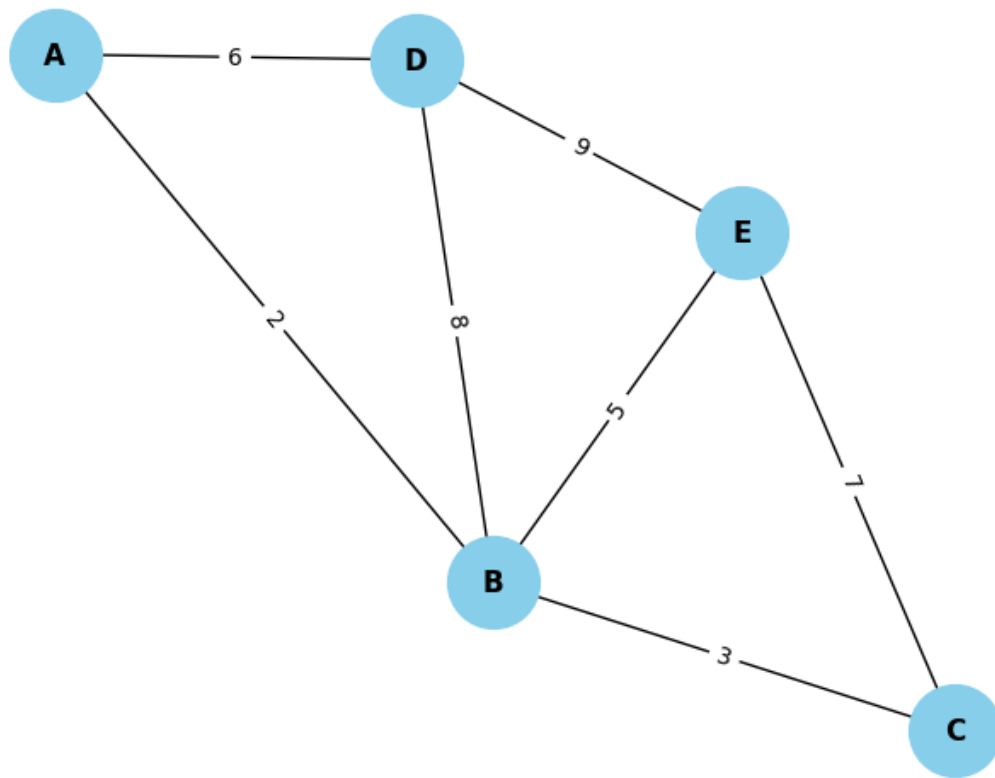
$$\omega_k d(v_k, x) = \omega_l d(v_l, x)$$

- EQ_e : Set of all equilibrium points on $e \in E$.
- EQ : Set of all equilibrium points on G .

2.2.2 Example

```
[13]: graph2 = [
      [0, 2, 0, 6, 0],
      [2, 0, 3, 8, 5],
      [0, 3, 0, 0, 7],
      [6, 8, 0, 0, 9],
      [0, 5, 7, 9, 0]
      ]
```

```
[14]: ShowGraph = Graph(graph2)
      ShowGraph.show()
```

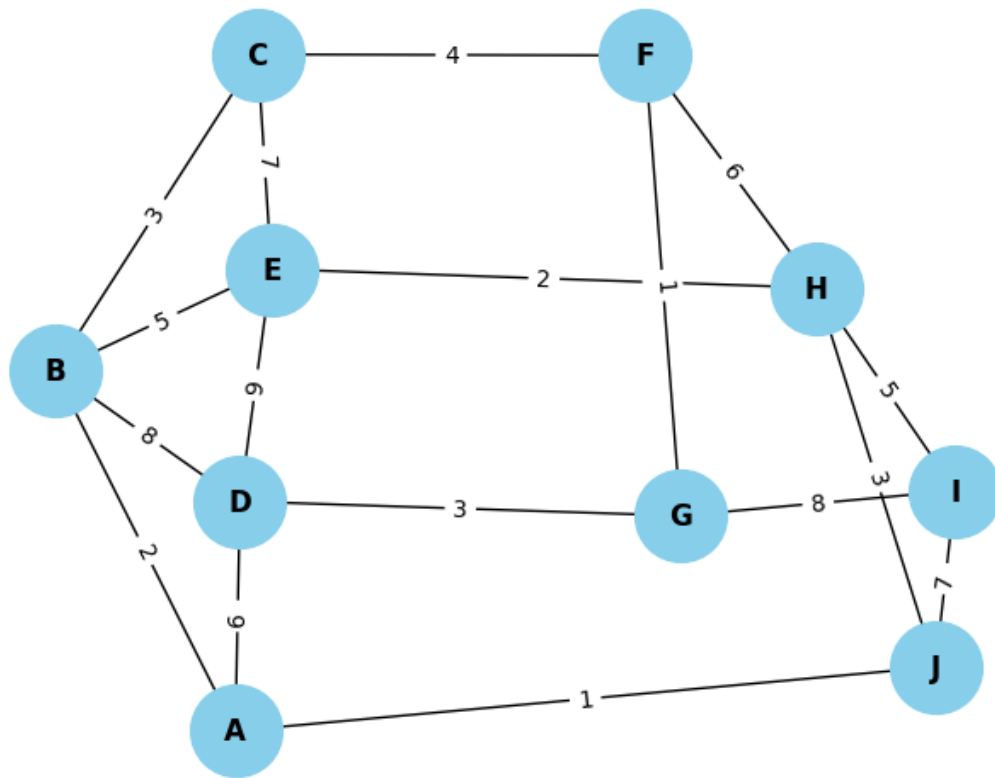


```
[15]: a = Local(graph2, 5, 1, 4)
      print(a.EquiPoint(3, 2))
```

0.4

```
[16]: graph3 = [
      [0, 2, 0, 6, 0, 0, 0, 0, 0, 1],
      [2, 0, 3, 8, 5, 0, 0, 0, 0, 0],
      [0, 3, 0, 0, 7, 4, 0, 0, 0, 0],
      [6, 8, 0, 0, 9, 0, 3, 0, 0, 0],
      [0, 5, 7, 9, 0, 0, 0, 2, 0, 0],
      [0, 0, 4, 0, 0, 0, 1, 6, 0, 0],
      [0, 0, 0, 3, 0, 1, 0, 0, 8, 0],
      [0, 0, 0, 0, 2, 6, 0, 0, 5, 3],
      [0, 0, 0, 0, 0, 0, 8, 5, 0, 7],
      [1, 0, 0, 0, 0, 0, 0, 3, 7, 0]
    ]
```

```
[17]: ShowGraph = Graph(graph3)
      ShowGraph.show()
```



```
[18]: a = Local(graph3, 10, 0, 6)
      print(a.EquiPoint(1, 5))
```

```
0.4444444444444444
```

2.3 Local center

2.3.1 Description

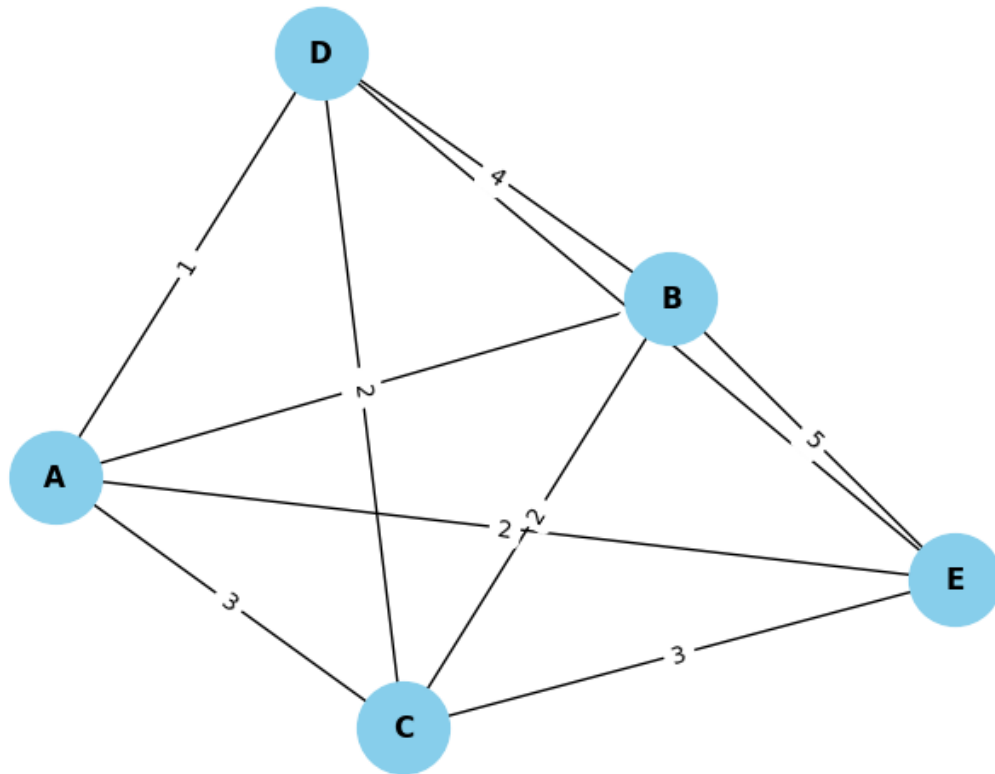
For $e \in E$, $e = (v_i, v_j)$, the local center on e is

$$\arg \min_{x \in e} f(x) = \arg \min \left\{ \min_{x \in EQ_e} f(x), f(v_i), f(v_j) \right\}$$

2.3.2 Bounded value

```
[19]: graph4 = [
      [0, 4, 3, 1, 2],
      [4, 0, 2, 4, 5],
      [3, 2, 0, 2, 3],
      [1, 4, 2, 0, 1],
      [2, 5, 3, 1, 0]
```

```
]
ShowGraph = Graph(graph4)
ShowGraph.show()
```



Upper bound $f(v_i)$

```
[20]: from packages.Point import LocalMinima
```

```
[21]: a = LocalMinima(graph4, 5, 0, 1)

print(
    a.lowerlm(2),
    a.lowerlm(3),
    a.lowerlm(4),
    a.lowerlm(1)
)
```

-10 -8 -7 -12

```
[22]: print(a.BoundedValue(x='upper'))
```

4

Lower bound $f(v_j)$

```
[23]: print(
      a.upperlm(1),
      a.upperlm(4),
      a.upperlm(2),
      a.upperlm(3)
    )
```

4 2 3 1

```
[24]: print(a.BoundedValue(x='lower'))
```

0

2.3.3 Interval value

$$\min_{x \in EQ_e} f(x)$$

The local minima on segment (A, D) (Not included lower and upper bound)

```
[25]: eg = LocalMinima(graph4, 5, 0, 1)

      min_val, min_x = eg.IntervalValue()
      print(min_x)
      print(min_val)
```

0.75

6.0

```
[26]: eg.show_lines(1.5)
```

Line 0: $f(1.5) = 9.0$

Line 1: $f(1.5) = 0.0$

Line 2: $f(1.5) = 7.0$

Line 3: $f(1.5) = 2.0$

Line 4: $f(1.5) = 8.0$

Line 5: $f(1.5) = 3.0$

Output value

2.4 Global center *[In progress...]*

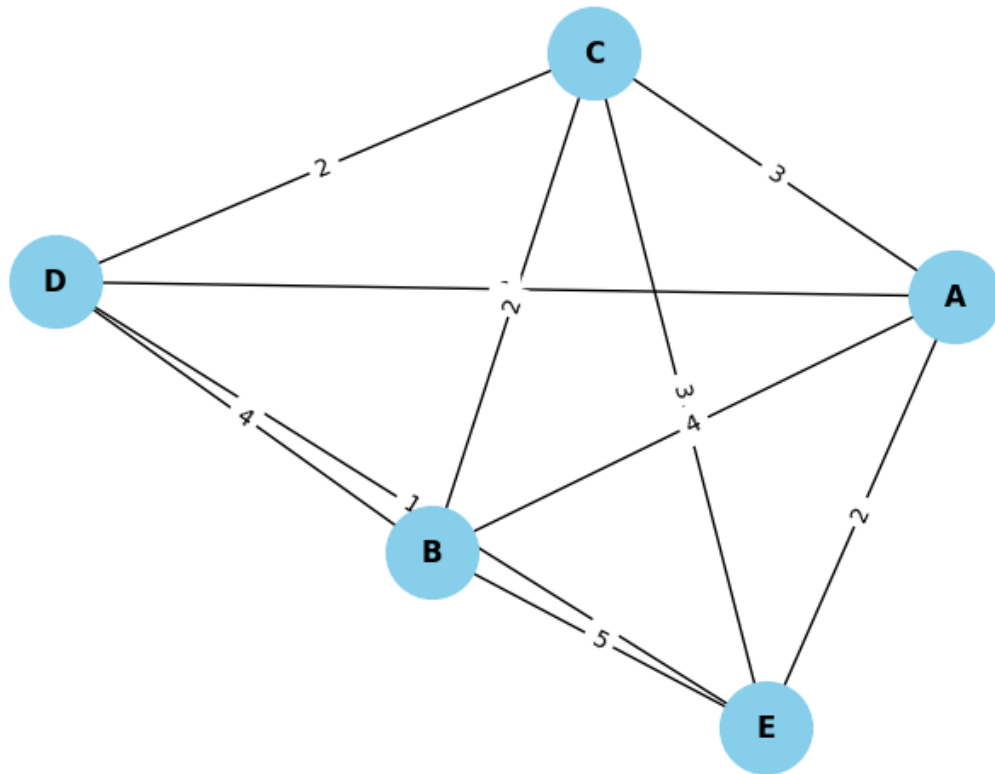
```
[27]: from packages.Point import GlobalCenter
```

```
[28]: Global = [
      [0, 4, 3, 1, 2],
      [4, 0, 2, 4, 5],
      [3, 2, 0, 2, 3],
      [1, 4, 2, 0, 1],
```

```

    [2, 5, 3, 1, 0]
]
ShowGraph = Graph(Global)
ShowGraph.show()

```



```
[29]: rs = GlobalCenter(Global, 5)
```

```
# rs.
```

```
rs.GlobalSearch()
```

```

Error with pair (0, 1): 'InLocalMin' object has no attribute 'LocalMinima'
Error with pair (0, 2): 'InLocalMin' object has no attribute 'LocalMinima'
Error with pair (0, 3): 'InLocalMin' object has no attribute 'LocalMinima'
Error with pair (0, 4): 'InLocalMin' object has no attribute 'LocalMinima'
Error with pair (1, 2): 'InLocalMin' object has no attribute 'LocalMinima'
Error with pair (1, 3): 'InLocalMin' object has no attribute 'LocalMinima'
Error with pair (1, 4): 'InLocalMin' object has no attribute 'LocalMinima'
Error with pair (2, 3): 'InLocalMin' object has no attribute 'LocalMinima'
Error with pair (2, 4): 'InLocalMin' object has no attribute 'LocalMinima'
Error with pair (3, 4): 'InLocalMin' object has no attribute 'LocalMinima'

```

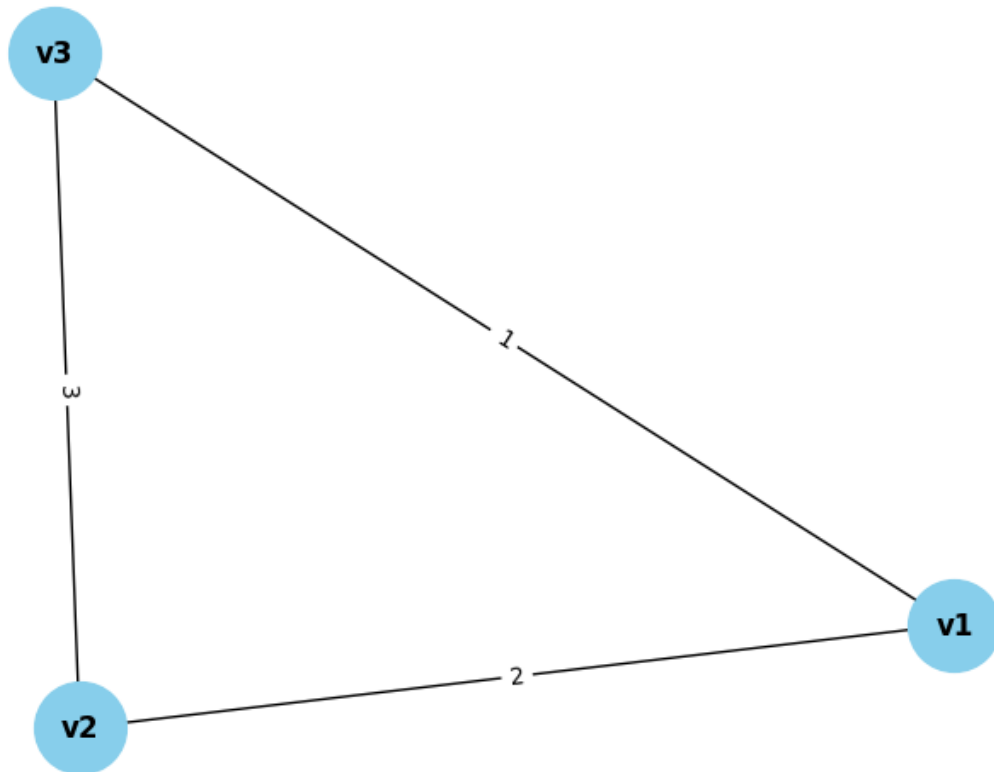
[29]: (inf, None)

3 Ordered Weber Region

```
[30]: from packages.Point import FunctionPlotter
```

Example

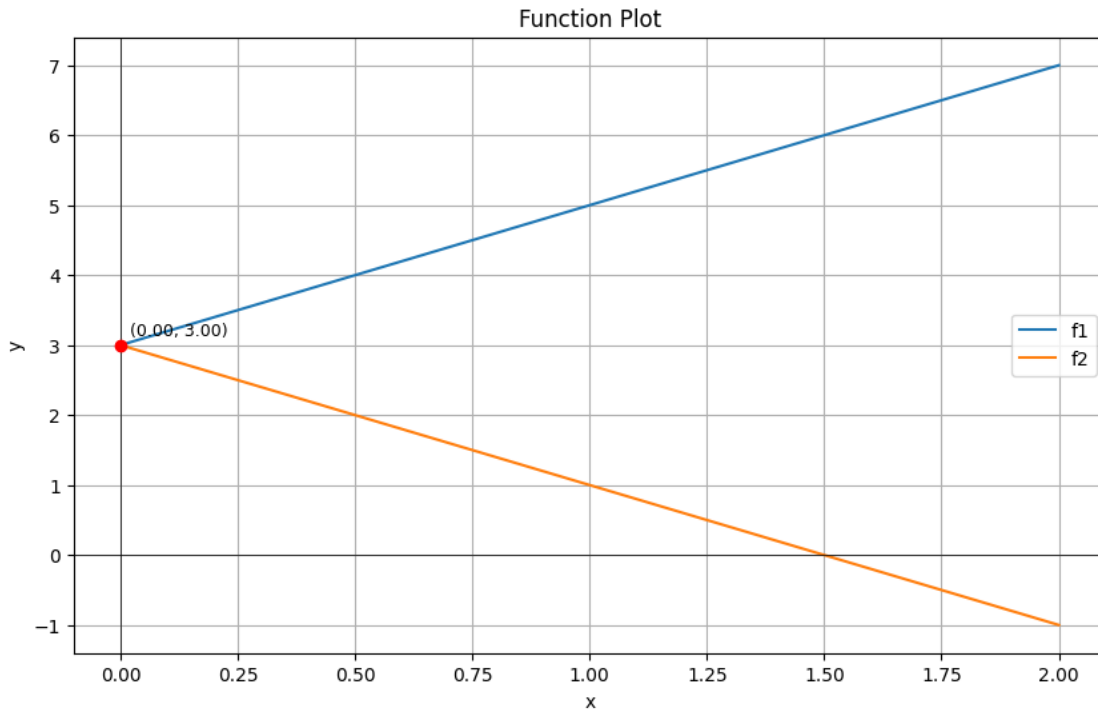
```
[31]: graph5 = [[0, 2, 1],  
                [2, 0, 3],  
                [1, 3, 0]  
                ]  
  
ord_weber = Graph(graph5)  
ord_weber.show_node()
```



$$x_1 = ((v_2, v_1), \alpha) = \begin{cases} f_1(x) &= 3 + 2x \\ f_2(x) &= 1 + (1 - x)2 \end{cases}$$

```
[32]: BA = FunctionPlotter(0, 2, 100)
def f1(x): return 3 + 2 * x
BA.add_function(f1, "f1")
def f2(x): return 1 + (1 - x) * 2
BA.add_function(f2, "f2")
BA.find_intersection(0, 1)
BA.plot()
```

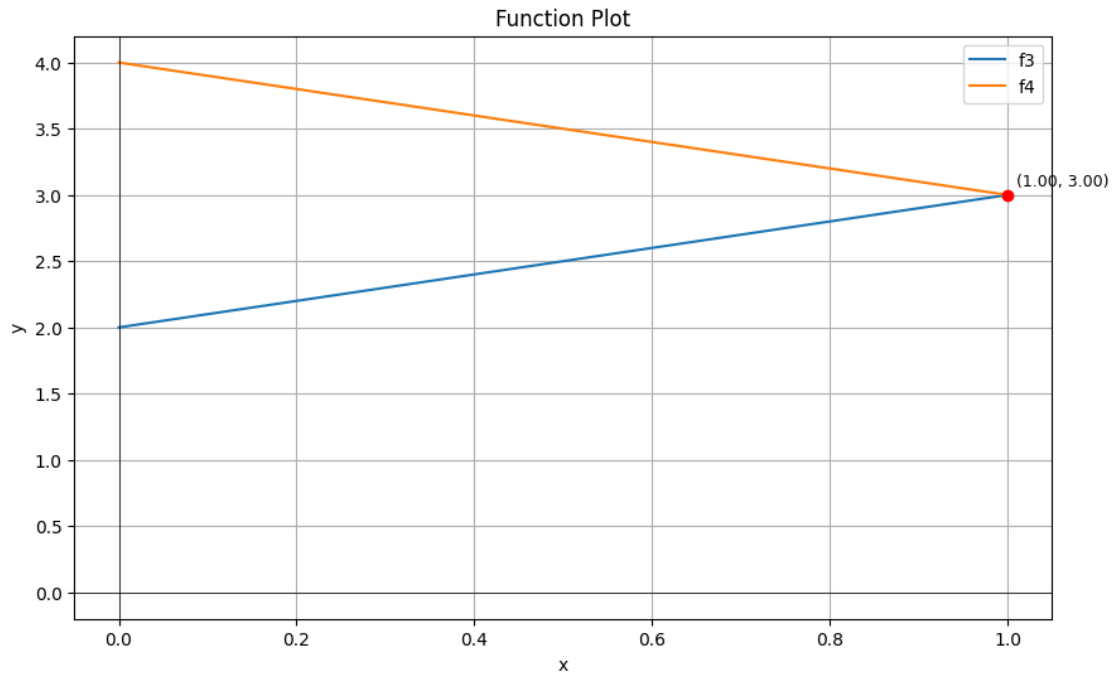
Intersection at x = 0.0000, y = 3.0000



$$x_2 = ((v_1, v_3), \alpha) = \begin{cases} f_3(x) & = 2 + \alpha \\ f_4(x) & = 3 + (1 - \alpha) \end{cases}$$

```
[33]: AC = FunctionPlotter(0, 1, 100)
def f3(x): return 2 + x
AC.add_function(f3, "f3")
def f4(x): return 3 + (1-x)
AC.add_function(f4, "f4")
AC.find_intersection(0, 1)
AC.plot()
```

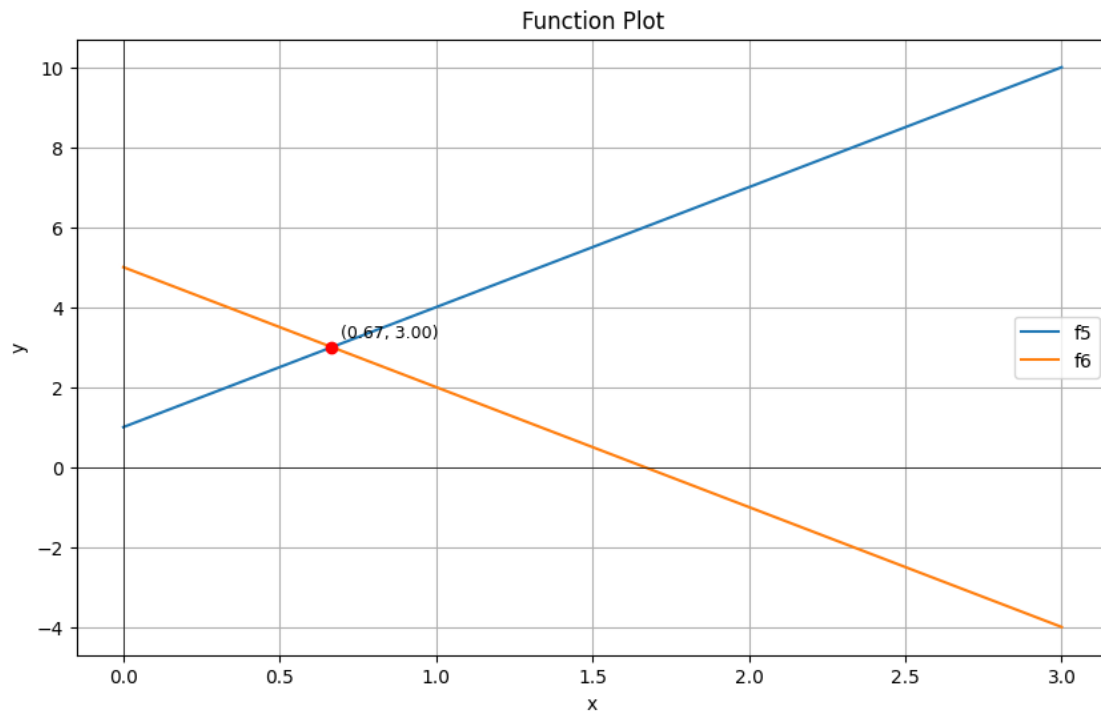
Intersection at x = 1.0000, y = 3.0000



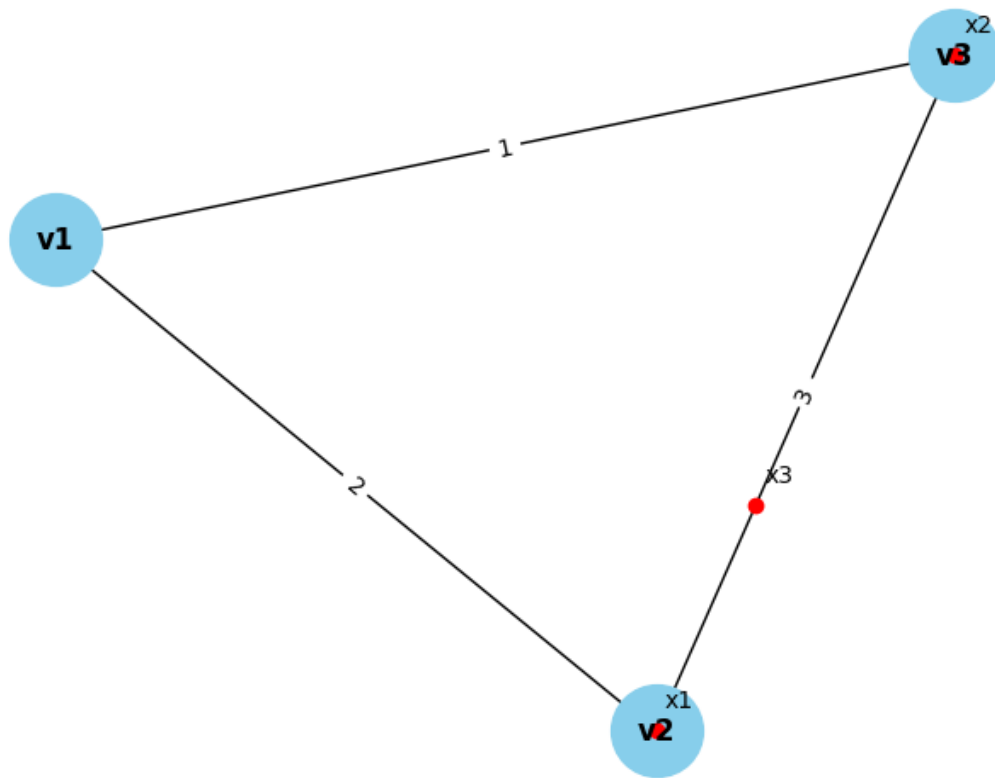
$$x_3 = ((v_3, v_2), \alpha) = \begin{cases} f_5(x) &= 1 + 3\alpha \\ f_6(x) &= 2 + (1 - \alpha)3 \end{cases}$$

```
[34]: CB = FunctionPlotter(0, 3, 100)
def f5(x): return 1 + 3*x
CB.add_function(f5, "f5")
def f6(x): return 2 + (1-x)*3
CB.add_function(f6, "f6")
CB.find_intersection(0, 1)
CB.plot()
```

Intersection at x = 0.6667, y = 3.0000

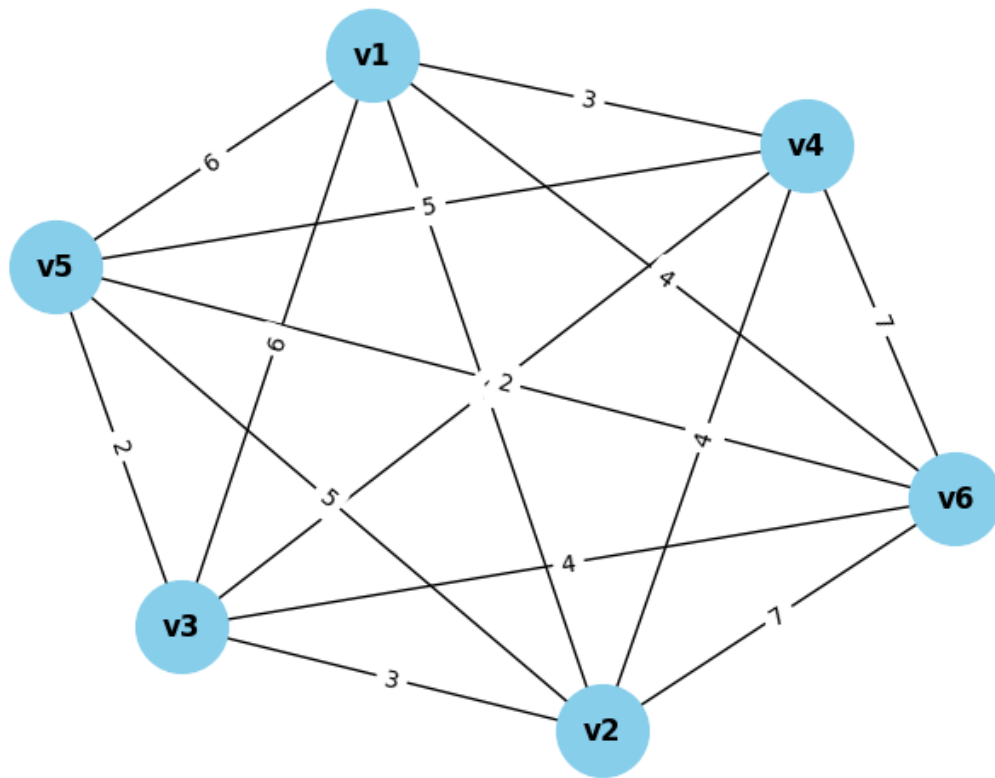


```
[37]: ord_weber.add_marked_point_on_edge('v2', 'v1', alpha=0, label='x1')
ord_weber.add_marked_point_on_edge('v1', 'v3', alpha=1, label='x2')
ord_weber.add_marked_point_on_edge('v3', 'v2', alpha=0.6667, label='x3')
ord_weber.show_node()
```



4 Example (Hakimi)

```
[35]: graph6 = [
    [0, 3, 6, 3, 6, 4],
    [3, 0, 3, 4, 5, 7],
    [6, 3, 0, 3, 2, 4],
    [3, 4, 3, 0, 5, 7],
    [6, 5, 2, 5, 0, 2],
    [4, 7, 4, 7, 2, 0]
]
Hakimi = Graph(graph6)
Hakimi.show_node()
```



```
[36]: v6_v5 = FunctionPlotter(0, 2, 100)
```

```
#  $v_i - v_6$ 
```

```
def v1v6(x): return 4 + 2*x
v6_v5.add_function(v1v6, "v1v6")
def v1v5(x): return 8 + (1-x)*2
v6_v5.add_function(v1v5, "v1v5")
```

```
def v2v6(x): return 7 + 2*x
v6_v5.add_function(v2v6, "v2v6")
def v2v5(x): return 5 + (1-x)*2
v6_v5.add_function(v2v5, "v2v5")
```

```
def v3v6(x): return 10 + 2*x
v6_v5.add_function(v3v6, "v3v6")
def v3v5(x): return 2 + (1-x)*2
v6_v5.add_function(v3v5, "v3v5")
```

```
def v4v6(x): return 7 + 2*x
```



```

v6_v5.add_function(v4v6, "v4v6")
def v4v5(x): return 5 + (1-x)*2
v6_v5.add_function(v4v5, "v4v5")

def v5v6(x): return x
v6_v5.add_function(v5v6, "v5v6")
def v6v5(x): return 2 - x
v6_v5.add_function(v6v5, "v6v5")

v6_v5.find_intersection(0, 1)
v6_v5.find_intersection(2, 3)
v6_v5.find_intersection(4, 5)
v6_v5.find_intersection(6, 7)
v6_v5.find_intersection(8, 9)

v6_v5.plot()

```

Intersection at $x = 1.5000$, $y = 7.0000$
 Intersection at $x = 0.0000$, $y = 7.0000$
 Intersection at $x = -1.5000$, $y = 7.0000$
 Intersection at $x = 0.0000$, $y = 7.0000$
 Intersection at $x = 1.0000$, $y = 1.0000$

